

PHASEMATRIX-HIVEMIND-03

Morphodynamisches Resonanzzell-Substrat

Formale Implementierungsspezifikation v0.3

Sebastian Klemm

Technische Ausarbeitung fuer Coding-Agent-Implementierung

Mai 2026

Zusammenfassung

Diese Spezifikation definiert die Version 0.3 des PhaseMatrix-Hivemind-Profiles als implementierungsbereites, domänenneutrales Substrat fuer morphodynamische Resonanzzellen. Version 0.3 ersetzt die PSE-Integrationsfassung v0.2 normativ, indem sie deren föderierte Phasenraum-, NullCenter-, Mirror-Seam-, Claim-, Lattice- und Truth-Maintenance-Struktur vollständig beibehält und eine fehlende Mikrostruktur ergänzt: PhaseCells, Gabriel-artige Resonatoren, TridentVector-Aktivierung, MorphodynamicField, ResonanceCluster, FunnelGraph, MandorlaConvergenceField, TensionToIntentOperator, OuroborosFeedback, ClusterLifecycle, trace-erhaltende Dissolution und Matrix-kompatible Subnet-Agentik. Militärische, Privacy-evasive und nicht belegte Sicherheitsclaims werden explizit ausgeschlossen. Der Kern ist ein allgemeines, auditierbares Zell- und Cluster-Modell fuer emergente Subnetze in föderierten post-symbolischen Kognitionssystemen.

Inhaltsverzeichnis

1	Status, Geltungsbereich und Ersatzbeziehung	3
1.1	Status	3
1.2	Ersatzbeziehung zu v0.2	3
1.3	Ziel	3
1.4	Nicht-Ziele	3
2	Neutralisierungspflicht	4
2.1	Ausgeschlossene Semantik	4
2.2	Erlaubte neutrale Strukturprimitive	4
3	Architekturposition	5
3.1	Gesamtstack	5
3.2	Schichtrollen	5
4	Mathematischer Kern	5
4.1	TridentVector	5
4.2	MorphodynamicField	6
4.3	ConvergenceField	6
5	Kanonische Primitive	6
5.1	Zahl- und Hashregeln	6
5.2	Basistypen	6
6	Run Descriptor v0.3	7
7	Datenmodelle: PhaseCell-Schicht	7
7.1	TridentVector	7
7.2	PhaseCell	8
7.3	CellPool	8
8	Gabriel-artige Resonatoren	9
8.1	LocalResonanceProcessor	9
8.2	ResonancePulse	9
9	MorphodynamicField	9
9.1	Datenstruktur	9
9.2	MorphologyEvent	10
10	ResonanceCluster	10
10.1	Cluster-Modell	10

10.2 ClusterFormationReport	11
11 FunnelGraph	11
11.1 Zweck	11
11.2 Datenstruktur	11
12 MandorlaConvergenceField	12
12.1 Datenstruktur	12
12.2 Konvergenzregel	12
13 TensionToIntentOperator	12
13.1 Zweck	12
13.2 Datenmodell	12
14 OuroborosFeedback	13
14.1 Bounded Feedback	13
14.2 Feedback-Regel	13
15 Dissolution und Trace-Erhaltung	13
15.1 Grundsatz	13
15.2 Datenmodell	14
16 ClusterTrace	14
17 Gates	14
17.1 Cluster Formation Gate	14
17.2 Morphology Gate	14
17.3 Intent Gate	14
17.4 Dissolution Gate	15
17.5 Matrix Boundary Gate	15
18 Pipeline v0.3	15
18.1 Referenzablauf	15
18.2 Pseudocode	15
19 Integration mit PhaseMatrix v0.2 und PSE	16
19.1 Matrix-Integration	16
19.2 PSE-Handoff	16
20 Outcome-Typen	16
21 CLI-Schnittstelle	17
21.1 Subcommands	17

21.2 CLI-Regeln	17
22 Feature Flags und Modulstruktur	17
22.1 Crate-Struktur	17
22.2 Feature Flags	18
23 Tests	18
23.1 Unit Tests	18
23.2 Integration Tests	19
23.3 Negative Tests	19
24 Definition of Done	19
25 Coding-Agent-Auftrag	20
26 Schlussformel	20

1 Status, Geltungsbereich und Ersatzbeziehung

1.1 Status

Dieses Dokument ist eine normative Implementierungsspezifikation. Die Begriffe **MUST**, **MUST NOT**, **SHOULD** und **MAY** sind verbindlich fuer Konformitaet.

1.2 Ersatzbeziehung zu v0.2

PHASEMATRIX-HIVEMIND-03 ersetzt PHASEMATRIX-HIVEMIND-02 als Zielprofil fuer Implementierung. v0.2 bleibt historisch nachvollziehbar, gilt aber als Teilmenge dieser Fassung.

Version	Rolle
v0.1	Allgemeines föderiertes Phasenraum-Kognitionssubstrat mit ADAMANT-Verfassung, NullCenter, Dual Fabric, Cross-Certification und Truth-Maintenance.
v0.2	PSE-Integrationsprofil mit PSE-Report-Import, Claim-Extraktion, Mirror-Seam-Pruefung, Lattice-Certification, Change-Set-Calculus, Matrix-Cycle und CLI.
v0.3	Vollstaendiges Implementierungsprofil: v0.2 plus morphodynamische Zell- und Cluster-Mikrostruktur, Gabriel-Resonatoren, Trident-Aktivierung, Mandorla-Konvergenz, Tension-to-Intent, FunnelGraph und trace-erhaltende Dissolution.

1.3 Ziel

Ziel ist eine universelle Mikrostruktur fuer PhaseMatrix-Subnetze. Ein Subnetz soll nicht nur aus statischen Nodes bestehen, sondern aus resonanzfaehigen PhaseCells, die sich zu temporären oder persistenten Arbeitsclustern bilden, ihre lokale Morphologie kontrolliert anpassen, Arbeitszustände kompaktieren, Claims erzeugen und dabei Trace, Evidence, Governance und Matrix-Gates erhalten.

Kernthese

Ein PhaseSubnet ist kein statisches Graphobjekt. Es ist ein kontrolliertes morphodynamisches Feld aus PhaseCells, ResonanceClusters, FunnelGraphs, ConvergenceFields und Lifecycle-Gates. Jede emergente Zellstruktur darf lokal arbeiten und sich wieder auflösen; ihre Gate-Entscheidungen, Evidence und Trace-Artefakte bleiben jedoch dauerhaft auditierbar.

1.4 Nicht-Ziele

Die Spezifikation ist nicht:

- eine militärische Command-and-Control-Architektur;
- ein UAV-, UGV-, UUV-, Waffen-, Payload- oder Einsatzsystem;
- ein Privacy-Evasion-, Tarnkommunikations- oder untraceable-networking-Protokoll;
- ein Kryptographiebeweis fuer Zero-Knowledge, Quantum-Resistance oder Forward-Secrecy;
- ein biologischer oder quantenphysikalischer Wirklichkeitsclaim;
- eine Erlaubnis, Audit-Spuren zu löschen.

2 Neutralisierungspflicht

2.1 Ausgeschlossene Semantik

Folgende Begriffe und Bedeutungen **MUST NOT** in API, CLI, Core-Reports oder Tests als operative Semantik erscheinen:

- `combat`, `battlefield`, `strike`, `SEAD`, `payload`, `special forces`, `stealth C2`;
- `untraceable`, `spurlos`, `no reconstruction possible`, `silent C2`, `low probability of detection`;
- `quantum-resistant` ohne formalen kryptographischen Beweis;
- `absolute privacy`, `perfect privacy`, `zero knowledge` ohne Beweissystem;
- medizinische, biologische, militärische oder physikalische Anwendungsclaims im Core.

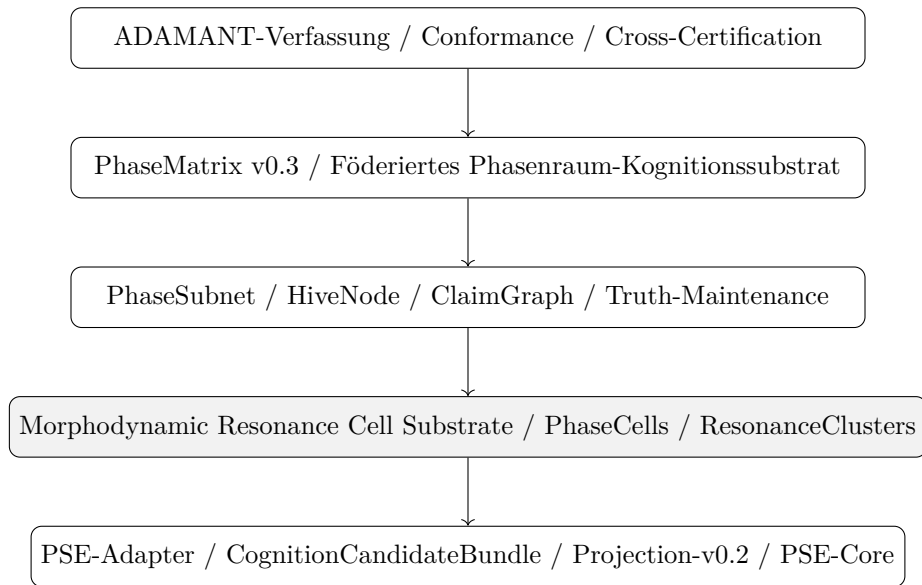
2.2 Erlaubte neutrale Strukturprimitive

Folgende neutralisierte Konzepte **MAY** als technische Primitive verwendet werden:

Quellbegriff	Neutraler Implementierungsbegriff
Gabriel Cell	<code>PhaseCell</code> oder <code>ResonanceCell</code> : lokaler semantischer Oszillator.
Gabriel Resonator	<code>LocalResonanceProcessor</code> : Eingabeprojektion auf <code>TridentVector</code> .
Mandorla Field	<code>ConvergenceField</code> : Überlappungs- und Stabilisierungsschicht.
HHP resonance channel	<code>ResonanceCoupling</code> : phasen- und kohärenzbasierte Aktivierung.
4D Funnel DAG	<code>FunnelGraph</code> : räumlich/zeitlich/semantisch/resonanter Traversalgraph.
Exkalibration	<code>EmissionTrigger</code> oder <code>IntentCandidate</code> .
Dissolution	<code>WorkingStateCompaction</code> : Arbeitszustand darf verschwinden; Trace bleibt.
Hyperbion growth	<code>MorphodynamicGrowth</code> : strukturelle Anpassung aus Phase und Wachstumspotential.

3 Architekturposition

3.1 Gesamtstack



3.2 Schichtrollen

Schicht	Rolle
Matrix-Verfassung	Konstitutionelle Regeln, Tracepflicht, Change-Sets, Proof-Obligations, Truth-Maintenance.
PhaseSubnet	Föderierte, lokal souveräne Teilmatrix mit Claims, Seams, Lattice und Governance.
CellPool	Vorrat an PhaseCells eines Nodes oder Subnetzes.
ResonanceCluster	Temporäre oder persistente Arbeitsgruppe aus PhaseCells.
FunnelGraph	Interne Cluster-Traversalstruktur ueber Raum/Zeit/Semantik/Resonanz.
ConvergenceField	Stabilisiert Zellpulse und erzeugt IntentCandidate-Vorstufen.
Dissolution	Kompaktiert Working State und erzeugt persistente Trace-Artefakte.

4 Mathematischer Kern

4.1 TridentVector

Jede PhaseCell und jeder ResonanceCluster besitzt einen normierten TridentVector:

$$T(t) = (s(t), c(t), p(t))$$

mit:

- $s(t)$: semantic density / Bedeutungsichte,
- $c(t)$: structural coherence / strukturelle Kohärenz,
- $p(t)$: temporal phase readiness / zeitliche Phasenbereitschaft.

Das Aktivierungspotential lautet:

$$D(t) = s(t) \cdot c(t) \cdot p(t).$$

Ein lokaler Trigger ist nur zulaessig, wenn:

$$D(t) \cdot L_T(t) > \theta_D$$

und alle Trace-, Gate- und Boundary-Bedingungen bestanden sind. $L_T(t)$ ist eine deterministische temporale Envelope- oder Horizon-Funktion.

4.2 MorphodynamicField

Die morphodynamische Wachstumsschicht ist:

$$H(x, t) = \alpha \cdot \Phi(x, t) + \beta \cdot \mu(x, t)$$

mit:

- $\Phi(x, t)$: phase resonance field,
- $\mu(x, t)$: morphodynamic growth field,
- α, β : run-descriptor-gebundene Gewichtungen.

H steuert nicht direkt Output. Es steuert nur Morphologieentscheidungen wie Grow, Split, Fuse, Decay, Stabilize oder WorkingStateCompaction.

4.3 ConvergenceField

Ein ConvergenceField akkumuliert Zellpulse:

$$C_{field}(t+1) = \text{Normalize} \left(\sum_i P_i(t) + M(t) \right)$$

mit P_i als Zellpuls und $M(t)$ als zugelassene Memory- oder Context-Projektion. Die Implementierung **MUST** eine fixed-point-kompatible, deterministische Normalisierung verwenden.

5 Kanonische Primitive

5.1 Zahl- und Hashregeln

- Alle gate-relevanten Werte **MUST** Fixed, CanonicalNumber oder rational normalisiert sein.
- Plattform-Floats **MUST NOT** in Audit-, Hash-, Gate- oder Replay-Pfaden vorkommen.
- Alle Maps **MUST** als BTreeMap oder aequivalente geordnete Struktur serialisiert werden.
- Alle Vektoren ohne semantische Reihenfolge **MUST** vor Hashing sortiert werden.
- Alle Reports **MUST** JCS- oder projektkanonisch serialisiert werden.

5.2 Basistypen

```
pub type Hash256 = [u8; 32];
pub type StableId = String;
pub type Fixed = CanonicalNumber;
```



```
pub struct EvidenceRef {
  pub kind: String,
  pub address: Hash256,
  pub relation: String,
}
```

6 Run Descriptor v0.3

```
pub struct PhaseMatrixRunDescriptorV3 {
  pub run_id: StableId,
  pub matrix_id: Hash256,
  pub parent_profile_hash: Option<Hash256>,
  pub operator_versions: BTreeMap<String, String>,
  pub canonicalization_version: String,
  pub thresholds: CellSubstrateThresholds,
  pub policies: CellSubstratePolicies,
  pub feature_flags: BTreeSet<String>,
  pub evidence_refs: Vec<EvidenceRef>,
}

pub struct CellSubstrateThresholds {
  pub min_cell_coherence: Fixed,
  pub min_cluster_coherence: Fixed,
  pub min_phase_alignment: Fixed,
  pub min_morphodynamic_potential: Fixed,
  pub max_cluster_conflict: Fixed,
  pub min_convergence_score: Fixed,
  pub min_intent_tension: Fixed,
  pub max_working_state_retention_ticks: u64,
}

pub struct CellSubstratePolicies {
  pub formation_policy: ClusterFormationPolicy,
  pub morphology_policy: MorphologyPolicy,
  pub dissolution_policy: DissolutionPolicy,
  pub trace_policy: ClusterTracePolicy,
  pub intent_policy: IntentPolicy,
  pub matrix_gate_policy: MatrixGatePolicy,
}
```

7 Datenmodelle: PhaseCell-Schicht

7.1 Trident Vector

```
pub struct TridentVector {
  pub semantic_density: Fixed,
  pub structural_coherence: Fixed,
  pub temporal_phase: Fixed,
}

impl TridentVector {
  pub fn activation_potential(&self) -> Fixed {
    // implemented with checked fixed-point multiplication
    self.semantic_density * self.structural_coherence * self.temporal_phase
  }
}
```

```
}
}
```

7.2 PhaseCell

```
pub struct PhaseCell {
    pub cell_id: Hash256,
    pub parent_node_id: Hash256,
    pub trident: TridentVector,
    pub local_resonance: Fixed,
    pub harmonization: Fixed,
    pub spiral_memory_state: Option<Hash256>,
    pub role: PhaseCellRole,
    pub lifecycle: CellLifecycle,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub enum PhaseCellRole {
    Sensor,
    Resonator,
    Router,
    Validator,
    MemoryProbe,
    BoundaryGuard,
    CandidateEmitter,
    MorphologyRegulator,
}

pub enum CellLifecycle {
    Dormant,
    Active,
    Clustered,
    Decaying,
    Compacted,
    Retired,
}
```

7.3 CellPool

```
pub struct CellPool {
    pub pool_id: Hash256,
    pub parent_subnet_id: Hash256,
    pub cells: Vec<PhaseCell>,
    pub pool_coherence: Fixed,
    pub pool_entropy: Fixed,
    pub active_cluster_refs: Vec<Hash256>,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

CellPool-Invariante

Ein CellPool **MUST** nur PhaseCells enthalten, deren `parent_node_id` oder delegierte Subnet-Autorisierung mit dem `parent_subnet_id` kompatibel ist. Fremde Zellen **MUST** vor Clusterbildung abgewiesen oder als ExternalEvidence markiert werden.

8 Gabriel-artige Resonatoren

8.1 LocalResonanceProcessor

```
pub struct LocalResonanceProcessor {
    pub processor_id: Hash256,
    pub input_projection_hash: Hash256,
    pub weight_profile_hash: Hash256,
    pub nonlinearity: ResonanceNonlinearity,
    pub output_trident: TridentVector,
    pub resonance_pulse: Fixed,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub enum ResonanceNonlinearity {
    Logistic,
    TanhApprox,
    SaturatingLinear,
    PiecewiseFixed,
}
```

8.2 ResonancePulse

```
pub struct ResonancePulse {
    pub pulse_id: Hash256,
    pub source_cell_id: Hash256,
    pub trident: TridentVector,
    pub activation_potential: Fixed,
    pub phase_bin: PhaseBin,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub enum PhaseBin {
    Continuous(Fixed),
    KPolar(u32),
    Tripolar(i8),
    Quadrupolar(i8, i8),
}
```

9 MorphodynamicField

9.1 Datenstruktur

```
pub struct MorphodynamicField {
    pub field_id: Hash256,
    pub parent_cluster_id: Option<Hash256>,
    pub phase_resonance_field: Fixed,
    pub growth_field: Fixed,
    pub alpha: Fixed,
    pub beta: Fixed,
    pub morphodynamic_potential: Fixed,
    pub endogeny_score: Fixed,
    pub exogeny_score: Fixed,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

```
}

```

9.2 MorphologyEvent

```
pub enum ClusterMorphologyEvent {
    Grow,
    Split,
    Fuse,
    Decay,
    Replicate,
    Stabilize,
    DissolveWorkingState,
    CompactToTrace,
}

pub struct MorphologyDecision {
    pub decision_id: Hash256,
    pub cluster_id: Hash256,
    pub event: ClusterMorphologyEvent,
    pub morphodynamic_potential: Fixed,
    pub gate_report_hash: Hash256,
    pub allowed: bool,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

10 ResonanceCluster

10.1 Cluster-Modell

```
pub struct ResonanceCluster {
    pub cluster_id: Hash256,
    pub parent_subnet_id: Hash256,
    pub member_cells: Vec<Hash256>,
    pub cluster_trident: TridentVector,
    pub coherence_score: Fixed,
    pub conflict_score: Fixed,
    pub morphodynamic_potential: Fixed,
    pub funnel_graph_hash: Hash256,
    pub convergence_field_hash: Hash256,
    pub lifecycle: ClusterLifecycle,
    pub trace_hash: Hash256,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub enum ClusterLifecycle {
    Proposed,
    Forming,
    Active,
    Stabilized,
    Splitting,
    Fusing,
    Decaying,
    Compacted,
    Dissolved,
}
```

```

    Rejected,
}

```

10.2 ClusterFormationReport

```

pub struct ClusterFormationReport {
    pub report_id: Hash256,
    pub pool_id: Hash256,
    pub proposed_member_cells: Vec<Hash256>,
    pub accepted_member_cells: Vec<Hash256>,
    pub rejected_member_cells: Vec<Hash256>,
    pub phase_alignment_score: Fixed,
    pub coherence_score: Fixed,
    pub morphodynamic_potential: Fixed,
    pub purpose_hash: Hash256,
    pub trace_ready: bool,
    pub passed: bool,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

11 FunnelGraph

11.1 Zweck

Der FunnelGraph ist keine verdeckte Routingstruktur. Er ist ein interner, auditierbarer Traversalgraph fuer einen ResonanceCluster. Er kann räumliche, zeitliche, semantische und resonante Kanten parallel halten.

11.2 Datenstruktur

```

pub struct FunnelGraph {
    pub graph_id: Hash256,
    pub cluster_id: Hash256,
    pub nodes: Vec<FunnelNode>,
    pub spatial_edges: Vec<FunnelEdge>,
    pub temporal_edges: Vec<FunnelEdge>,
    pub semantic_edges: Vec<FunnelEdge>,
    pub resonance_edges: Vec<FunnelEdge>,
    pub acyclic: bool,
    pub trace_hash: Hash256,
}

pub struct FunnelNode {
    pub node_id: Hash256,
    pub cell_id: Option<Hash256>,
    pub node_kind: FunnelNodeKind,
    pub state_hash: Hash256,
}

pub struct FunnelEdge {
    pub edge_id: Hash256,
    pub source: Hash256,
    pub target: Hash256,
}

```

```

    pub weight: Fixed,
    pub edge_kind: FunnelEdgeKind,
}

pub enum FunnelNodeKind { Cell, Pulse, Memory, Boundary, Intent, Trace }
pub enum FunnelEdgeKind { Spatial, Temporal, Semantic, Resonance, Evidence }

```

FunnelGraph-Invariante

Wenn `acyclic=true`, **MUST** der Graph vor Hashing topologisch validiert werden. Zyklische Graphen **MAY** nur verwendet werden, wenn der `RunDescriptor` explizit `cyclic_funnel_allowed=true` setzt und eine Replay-sichere `CyclePolicy` deklariert.

12 MandorlaConvergenceField

12.1 Datenstruktur

```

pub struct MandorlaConvergenceField {
    pub field_id: Hash256,
    pub cluster_id: Hash256,
    pub input_pulses: Vec<Hash256>,
    pub memory_projection_hash: Option<Hash256>,
    pub convergence_score: Fixed,
    pub conflict_score: Fixed,
    pub stabilized_intent_hash: Option<Hash256>,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

12.2 Konvergenzregel

Ein `ConvergenceField` ist stabil, wenn:

$$G_{conv} = 1 \iff score_{conv} \geq \theta_{conv} \wedge score_{conflict} \leq \theta_{conflict} \wedge trace_ready = 1.$$

13 TensionToIntentOperator

13.1 Zweck

Der `TensionToIntentOperator` wandelt keine Feldspannung in direkte Aktion um. Er erzeugt einen `IntentCandidate`, der durch Matrix-Gates, Claims, Truth-Maintenance und gegebenenfalls PSE-Handoff weiter validiert werden muss.

13.2 Datenmodell

```

pub struct TensionToIntentOperator {
    pub operator_id: Hash256,
    pub cluster_id: Hash256,
    pub field_tension: Fixed,
    pub convergence_score: Fixed,
    pub intent_threshold: Fixed,
    pub intent_candidate_hash: Option<Hash256>,
}

```

```

    pub passed: bool,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub struct IntentCandidate {
    pub candidate_id: Hash256,
    pub source_cluster_id: Hash256,
    pub purpose_hash: Hash256,
    pub intent_vector_hash: Hash256,
    pub confidence: Fixed,
    pub claim_refs: Vec<Hash256>,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

14 OuroborosFeedback

14.1 Bounded Feedback

OuroborosFeedback ist eine begrenzte, replayfähige Rückkopplung fuer Zell- und Clusterzustände. Sie darf keine unkontrollierte Selbstmodifikation erzeugen.

```

pub struct OuroborosFeedbackReport {
    pub report_id: Hash256,
    pub target_id: Hash256,
    pub previous_state_hash: Hash256,
    pub trident_response_hash: Hash256,
    pub learning_rate: Fixed,
    pub delta_hash: Hash256,
    pub new_state_hash: Hash256,
    pub bounded: bool,
    pub passed: bool,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

14.2 Feedback-Regel

$$S(t) = S(t - 1) + \eta \cdot f(T(t))$$

ist nur zulaessig, wenn:

$$||S(t) - S(t - 1)|| \leq \Delta_{max}$$

und alle Change-Set-/Conformance-Regeln der Matrix erfüllt sind.

15 Dissolution und Trace-Erhaltung

15.1 Grundsatz

Dissolution-Grundsatz

Ein Arbeitszustand **MAY** kompaktieren, verfallen oder aus dem aktiven Speicher entfernt werden. Gate-Entscheidungen, EvidenceRefs, ClusterTrace, Claim-Verweise, Lifecycle-Events und Hashes **MUST** erhalten bleiben. Die Spezifikation erlaubt keine spurenlose Auflösung.

15.2 Datenmodell

```
pub enum DissolutionMode {
    DropWorkingState,
    CompactToTrace,
    PersistEvidenceOnly,
    PersistClusterSummary,
    ArchiveFullState,
}

pub struct DissolutionReport {
    pub report_id: Hash256,
    pub cluster_id: Hash256,
    pub mode: DissolutionMode,
    pub working_state_before_hash: Hash256,
    pub retained_trace_hash: Hash256,
    pub retained_evidence_hashes: Vec<Hash256>,
    pub lifecycle_event_hashes: Vec<Hash256>,
    pub passed: bool,
}
```

16 ClusterTrace

```
pub struct ClusterTrace {
    pub trace_id: Hash256,
    pub cluster_id: Hash256,
    pub formation_report_hash: Hash256,
    pub funnel_graph_hash: Hash256,
    pub convergence_field_hash: Hash256,
    pub morphology_decision_hashes: Vec<Hash256>,
    pub intent_candidate_hashes: Vec<Hash256>,
    pub dissolution_report_hash: Option<Hash256>,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

17 Gates

17.1 Cluster Formation Gate

$$G_{cluster} = 1 \iff G_{phase} = 1 \wedge score_{coh} \geq \theta_{coh} \wedge H \geq \theta_H \wedge purpose_valid = 1 \wedge trace_ready = 1.$$

17.2 Morphology Gate

$$G_{morph} = 1 \iff score_{endo} \geq \theta_{endo} \wedge score_{exo} \geq \theta_{exo} \wedge lifecycle_allows(event) = 1 \wedge boundary_safe = 1.$$

17.3 Intent Gate

$$G_{intent} = 1 \iff field_tension \geq \theta_{tension} \wedge score_{conv} \geq \theta_{conv} \wedge score_{conflict} \leq \theta_{conflict} \wedge trace_ready = 1.$$

17.4 Dissolution Gate

$$G_{dissolve} = 1 \iff working_state_eligible = 1 \wedge trace_persisted = 1 \wedge evidence_persisted = 1 \wedge gate_history_p$$

17.5 Matrix Boundary Gate

Kein Clusterereignis darf Matrix-Invarianten verletzen:

$$G_{matrix} = G_{claim} \wedge G_{truth} \wedge G_{seam} \wedge G_{lattice} \wedge G_{governance}.$$

18 Pipeline v0.3

18.1 Referenzablauf

1. Lade PhaseMatrixRunDescriptorV3.
2. Lade PhaseSubnet, HiveNode und optional PSE-/Cognition-Reports.
3. Baue oder aktualisiere CellPool.
4. Führe LocalResonanceProcessor fuer relevante Inputs aus.
5. Berechne TridentVectors und ResonancePulses.
6. Evaluiere ClusterFormationGate.
7. Erzeuge bei Gate-Pass ResonanceCluster; sonst ClusterHoldReport.
8. Baue FunnelGraph.
9. Aktualisiere MorphodynamicField.
10. Evaluiere MorphologyGate und wende erlaubte MorphologyEvents an.
11. Aggregiere Pulse im MandorlaConvergenceField.
12. Evaluiere TensionToIntentOperator.
13. Erzeuge IntentCandidate nur bei Gate-Pass.
14. Schreibe ClusterTrace.
15. Kompaktiere Working State gemaess DissolutionPolicy.
16. Übergib validierte IntentCandidates als MatrixClaims oder PSE-Handoff-Kandidaten.

18.2 Pseudocode

```
pub fn run_cell_substrate_cycle(
  rd: PhaseMatrixRunDescriptorV3,
  input: CellSubstrateInput,
) -> CellSubstrateOutcome {
  let pool = build_or_update_cell_pool(&rd, &input);
  let pulses = compute_resonance_pulses(&rd, &pool, &input);
  let formation = evaluate_cluster_formation(&rd, &pool, &pulses);

  if !formation.passed {
    return CellSubstrateOutcome::Hold(make_cluster_hold(formation));
  }

  let mut cluster = form_resonance_cluster(&rd, &pool, &formation);
  let funnel = build_funnel_graph(&rd, &cluster, &pulses);
  let morph = compute_morphodynamic_field(&rd, &cluster, &funnel);
  let morph_decision = evaluate_morphology_gate(&rd, &cluster, &morph);
```

```

cluster = apply_allowed_morphology_event(&rd, cluster, morph_decision);

let convergence = update_convergence_field(&rd, &cluster, &pulses);
let intent = evaluate_tension_to_intent(&rd, &cluster, &convergence);
let trace = write_cluster_trace(&rd, &cluster, &funnel, &convergence, &intent);
let dissolution = maybe_compact_working_state(&rd, &cluster, &trace);

CellSubstrateOutcome::Completed(CellSubstrateCycleReport {
    cluster,
    funnel,
    convergence,
    intent,
    trace,
    dissolution,
})
}

```

19 Integration mit PhaseMatrix v0.2 und PSE

19.1 Matrix-Integration

Ein **IntentCandidate** **MAY** in einen **MatrixClaim** übersetzt werden, wenn:

- der **ClusterTrace** vollständig ist;
- der **IntentGate** bestanden ist;
- keine **Truth-Maintenance-Verletzung** vorliegt;
- **MatrixBoundaryGate** bestanden ist;
- **ClaimEvidence** vollständig content-addressed ist.

19.2 PSE-Handoff

Ein **IntentCandidate** **MAY** als **PSE-Handoff-Kandidat** dienen, aber **MUST NOT** direkt **SemanticCrystal**, **FinalizedEmission** oder **PSE-Commit** erzeugen.

```

pub struct CellToPseHandoffCandidate {
    pub handoff_id: Hash256,
    pub intent_candidate_id: Hash256,
    pub cluster_trace_id: Hash256,
    pub matrix_claim_refs: Vec<Hash256>,
    pub recommended_pse_adapter: Option<String>,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

20 Outcome-Typen

```

pub enum CellSubstrateOutcome {
    Completed(CellSubstrateCycleReport),
    Hold(ClusterHoldReport),
    Rejected(ClusterRejectionReport),
    Compacted(DissolutionReport),
    MatrixBoundaryViolation(MatrixBoundaryReport),
    DeterminismViolation(DeterminismReport),
}

```

```

}

pub struct CellSubstrateCycleReport {
    pub report_id: Hash256,
    pub rd_hash: Hash256,
    pub pool_hash: Hash256,
    pub cluster: ResonanceCluster,
    pub funnel: FunnelGraph,
    pub convergence: MandorlaConvergenceField,
    pub intent: Option<IntentCandidate>,
    pub trace: ClusterTrace,
    pub dissolution: Option<DissolutionReport>,
}

```

21 CLI-Schnittstelle

21.1 Subcommands

```

phase-matrix cell-pool <subnet.json> --rd rd.json --out pool.json
phase-matrix cell-resonate <pool.json> --input input.json --rd rd.json --out pulses.json
phase-matrix cluster-form <pool.json> --pulses pulses.json --rd rd.json --out cluster.
    json
phase-matrix funnel-build <cluster.json> --rd rd.json --out funnel.json
phase-matrix morph-step <cluster.json> --funnel funnel.json --rd rd.json --out morph.
    json
phase-matrix converge <cluster.json> --pulses pulses.json --rd rd.json --out convergence.
    json
phase-matrix intent <convergence.json> --cluster cluster.json --rd rd.json --out intent.
    json
phase-matrix cluster-dissolve <cluster.json> --trace trace.json --rd rd.json --out
    dissolution.json
phase-matrix cluster-cycle <subnet.json> --input input.json --rd rd.json --out cycle.
    json
phase-matrix cluster-replay <cycle.json>
phase-matrix cluster-verify <trace.json>

```

21.2 CLI-Regeln

- **cluster-cycle** **MUST** die gesamte Pipeline ausführen und ein content-addressed Report erzeugen.
- **cluster-dissolve** **MUST** Trace-Erhaltung prüfen und bei fehlender Trace-Erhaltung fehlschlagen.
- **cluster-replay** **MUST** byte-identische Reproduktion aller IDs und Reports prüfen.
- **cluster-verify** **MUST** Formation, FunnelGraph, Convergence, Intent und Dissolution referentiell prüfen.

22 Feature Flags und Modulstruktur

22.1 Crate-Struktur

```

crates/phase-matrix/src/cell/
  mod.rs
  primitives.rs
  trident_vector.rs
  phase_cell.rs
  cell_pool.rs
  local_resonance_processor.rs
  resonance_pulse.rs
  morphodynamic_field.rs
  resonance_cluster.rs
  funnel_graph.rs
  convergence_field.rs
  tension_to_intent.rs
  ouroboros_feedback.rs
  dissolution.rs
  cluster_trace.rs
  cluster_gate.rs
  pse_handoff.rs
  report.rs
  replay.rs
  pipeline.rs

```

22.2 Feature Flags

Feature	Bedeutung
cell-substrate	Aktiviert Kern der morphodynamischen Zellschicht.
cell-cli	Baut CLI-Erweiterungen.
cell-funnel-graph	Aktiviert FunnelGraph-Konstruktion und Validierung.
cell-morphodynamics	Aktiviert MorphodynamicField und MorphologyEvents.
cell-convergence	Aktiviert MandorlaConvergenceField und IntentOperator.
cell-pse-handoff	Aktiviert Übergabe an PSE-Adapter; erzeugt keinen Commit.

23 Tests

23.1 Unit Tests

1. trident_activation_is_deterministic
2. phase_cell_hash_is_stable
3. cell_pool_sorts_cells_before_hashing
4. resonance_processor_rejects_nonfinite_values
5. cluster_forms_only_above_thresholds
6. cluster_gate_fails_closed_without_trace
7. morphodynamic_potential_matches_formula
8. morphology_gate_blocks_boundary_violation
9. funnel_graph_acyclic_validation_passes
10. funnel_graph_cycle_rejected_by_default
11. convergence_field_requires_low_conflict

12. `intent_gate_requires_tension_and_convergence`
13. `dissolution_preserves_trace`
14. `dissolution_preserves_evidence`
15. `cluster_trace_is_content_addressed`
16. `pse_handoff_does_not_create_crystal_or_finalized_emission`

23.2 Integration Tests

1. `cell_cycle_produces_report_or_hold`
2. `cell_cycle_replay_byte_identical`
3. `cluster_dissolve_keeps_trace_after_compaction`
4. `funnel_to_convergence_to_intent_happy_path`
5. `matrix_boundary_violation_blocks_intent_claim`
6. `pse_handoff_candidate_contains_cluster_trace`
7. `cli_cluster_cycle_writes_report`
8. `cli_cluster_replay_ok`
9. `cli_cluster_verify_detects_modified_trace`

23.3 Negative Tests

1. API darf keine Military-/Combat-/Payload-Typen exportieren.
2. API darf keine Privacy-Evasion-Claims erzeugen.
3. Dissolution darf Working State entfernen, aber niemals Trace oder Evidence entfernen.
4. Cluster darf MatrixBoundaryGate nicht umgehen.
5. Cluster darf keinen PSE-Commit erzeugen.

24 Definition of Done

Eine Implementierung von PHASEMATRIX-HIVEMIND-03 ist abgeschlossen, wenn:

1. `cargo test -workspace` besteht.
2. Alle neuen öffentlichen Typen dokumentiert sind.
3. Der Zellkern ohne Netzwerkabhängigkeit kompiliert.
4. Kein militärischer oder Privacy-evasiver Begriff in public API, CLI oder Report-Schema vorkommt.
5. Alle gate-relevanten Werte fixed-point oder kanonisch rational sind.
6. Alle Reports content-addressed und replayfähig sind.
7. Clusterbildung fail-closed ist.
8. FunnelGraph topologisch validiert wird.
9. MorphologyEvents nur durch MorphologyGate erlaubt werden.
10. IntentCandidates nur durch IntentGate entstehen.
11. Dissolution Trace, Evidence und Gate-Historie erhält.
12. PSE-Handoff keine `SemanticCrystal` oder `FinalizedEmission` erzeugt.
13. CLI `cluster-cycle`, `cluster-replay` und `cluster-verify` end-to-end funktionieren.
14. Golden Fixtures fuer Happy Path, Hold, BoundaryViolation und Dissolution existieren.

25 Coding-Agent-Auftrag

PATCH-ID: PHASEMATRIX-HIVEMIND-03

TITLE: Morphodynamic Resonance Cell Substrate

GOAL:

Implementiere eine morphodynamische Zell- und Cluster-Mikrostruktur fuer phase-matrix. v0.3 ersetzt v0.2 normativ, indem v0.2-Matrix-, Claim-, Truth-, Seam-, Lattice- und PSE-Integrationslogik erhalten bleibt und durch PhaseCells, TridentVector, LocalResonanceProcessor, ResonanceCluster, MorphodynamicField, FunnelGraph, ConvergenceField, TensionToIntent, OuroborosFeedback, Dissolution und ClusterTrace ergaenzt wird.

MUST:

- Module unter crates/phase-matrix/src/cell/ anlegen;
- alle Typen dieser Spezifikation implementieren;
- keine platform-floats im Audit-/Gate-/Hash-Pfad verwenden;
- BTreeMap/BTreeSet und sortierte Vektoren vor Hashing nutzen;
- ClusterFormationGate, MorphologyGate, IntentGate, DissolutionGate und MatrixBoundaryGate fail-closed implementieren;
- Working State Compaction erlauben, aber Trace/Evidence/Gate-History erhalten;
- CLI-Subcommands cell-pool, cell-resonate, cluster-form, funnel-build, morph-step, converge, intent, cluster-dissolve, cluster-cycle, cluster-replay und cluster-verify implementieren;
- Golden Fixtures und Tests gemaess Testplan liefern;
- PSE-Handoff nur als Kandidat erzeugen, ohne Commit/Crystal/FinalizedEmission.

MUST NOT:

- Military/C2/Combat/Payload/Stealth/Untraceable/Zero-Knowledge/Quantum-Resistant Claims oder Typen in den Core uebernehmen;
- Trace oder Evidence durch Dissolution entfernen;
- MatrixBoundaryGate umgehen;
- PSE-Core-Commits erzeugen;
- nichtdeterministische Randomness im Audit-Pfad nutzen.

SHOULD:

- v0.2-Kompatibilitaet ueber Re-Exports oder Adapter erhalten;
- Feature Flags cell-substrate, cell-cli, cell-funnel-graph, cell-morphodynamics, cell-convergence und cell-pse-handoff bereitstellen;
- klare README-Sektion zur Neutralisierung von SSCC/HHP/QFI/Hyperbion-Semantik schreiben.

26 Schlussformel

$$CellSubstrate(s) = IntentCandidate$$

nur wenn $G_{cluster} = 1 \wedge G_{morph} = 1 \wedge G_{conv} = 1 \wedge G_{intent} = 1 \wedge G_{matrix} = 1 \wedge TraceReady = 1$.

Andernfalls gilt:

$$CellSubstrate(s) = Hold(s, diagnostics).$$

Endinvariante

Die PhaseMatrix darf emergente Zellcluster erzeugen, transformieren, fusionieren, teilen und kompaktieren. Sie darf jedoch niemals ihre eigene Nachvollziehbarkeit verlieren. Emergenz ist erlaubt; nicht-auditierbare Emergenz ist verboten.
